Tutorial 5

Interpreter

Programming Languages & Compilers Spring 2025

School of Computer Science, Wuhan University

Speaker: Yang Yuanzhao

Tutorial Description

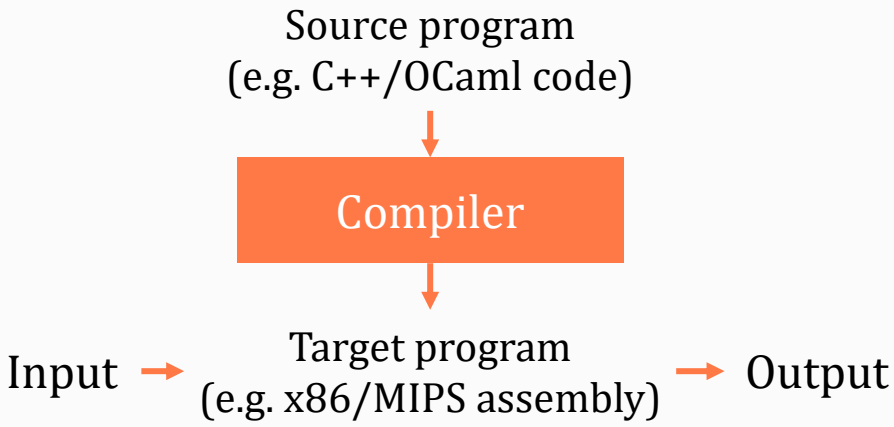
- This tutorial will first walk you through the process of interpreting simple mathematical expressions. And then we will implement a more complex interpreter for SimPL, a very simple programming language, and lambda calculus.
- The code of this tutorial is available in GitHub Repo **AugustineYang/OCaml-Interpreter**.
- The branch *calculator* and the release "**Math Interpreter**" contains the source code of a basic interpreter for simple mathematical expressions, which will be used as examples in the following sections.
- You are expected to finish your SimPL interpreter **on your own** based on the release "**SimPL Interpreter – Dev-Ready**". The complete implementation is available in repo's branch *SimPL* and releases.
- You are expected to finish your lambda calculus interpreter **on your own** based on your own SimPL interpreter. The complete implementation is available in branch *lambda* and releases.

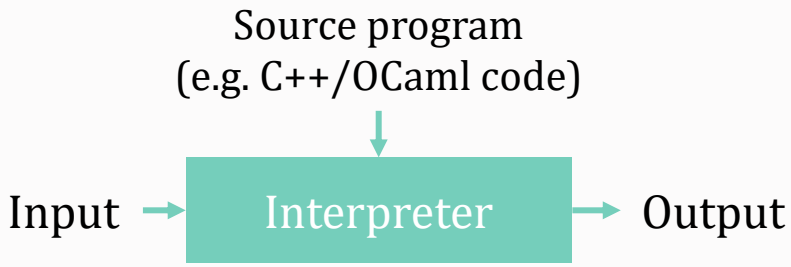
PART ONE

01 Introduction

Compiler vs. Interpreter

- A ***compiler*** is a program that implements a programming language. Its primary task is **translation**. The source program is typically expressed in a **high-level language,** and the target program is typically expressed in a **low-level language**. And the compiler is no longer needed after translation. Compilers target on **better performance.**

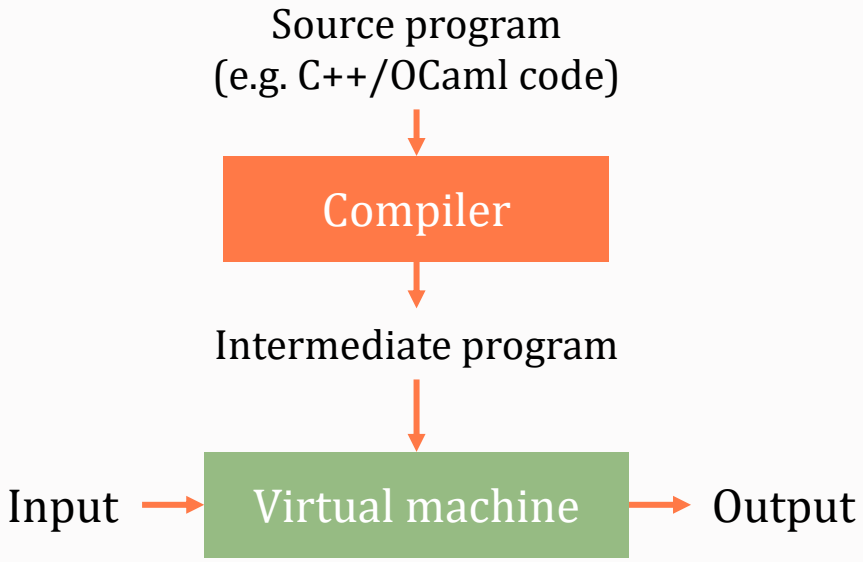




- An ***interpreter*** is a program that implements a programming language as well. However, its primary task is **execution**. It takes a source program and **directly executes** it without producing any target program. Interpreters target on **easier implementation**.

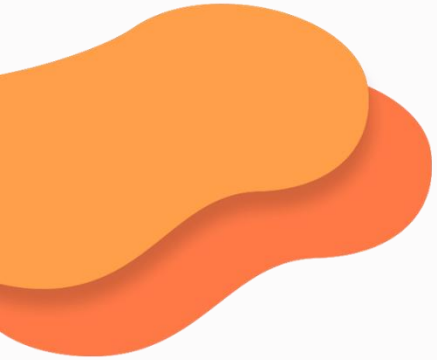
Compiler vs. Interpreter

- It is also possible to implement a language using a mixture of compilation and interpretation, such as many virtual machines like Java Virtual Machine (JVM), OCaml virtual machine, etc. And just-in-time compilation (JIT) takes a step further for better performance.



Interpreter

- An interpreter just works like the front end of a compiler, i.e., it does lexing, parsing and semantic analysis just as a compiler does. It then executes the AST right after that.
- In other words, an interpreter has the working phases as followed:
  - Lexing and Parsing
  - Type reconstruction (type inference and type checking)
  - Evaluation
- **Type reconstruction** is performed during semantic analysis phase. **Evaluation** happens when executing the program, or AST more precisely.



PART TWO

02 Lexing and Parsing

Lexing and Parsing

- Lexer and Parser work exactly as described in Tutorial 4. You can review the code in ast.ml, lexer.mll and parser.mly. Some definitions are slightly different from the ones in tutorial 4, preparing for extending to SimPL. You're expected to figure out what's happening without difficulty.

Definitions in ast.ml

```
type binop =
  | Add
  | Sub
  | Mul
  | Div

type expr =
  | Int of int
  | Binop of binop * expr * expr
```

Declarations in parser.mll

```
%token <int> INT
%token PLUS MINUS TIMES DIV EOF
%token LPAREN RPAREN

%left PLUS MINUS
%left TIMES DIV

%start main
%type <Ast.expr> main
%%
;
```

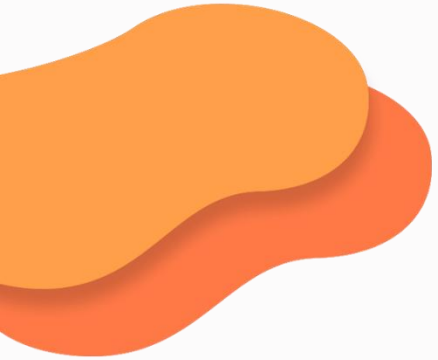
Rules in parser.mll

```
main:
    expr EOF { $1 }
;

expr:
    | INT { Int $1 }
    | expr TIMES expra { Binop (Mul, $1,
$3) }
    | expr DIV expr   { Binop (Div, $1, $3) }
    | expr PLUS expr  { Binop (Add, $1, $3) }
    | expr MINUS expr { Binop (Sub, $1, $3) }
    | LPAREN expr RPAREN { $2 }
;
```

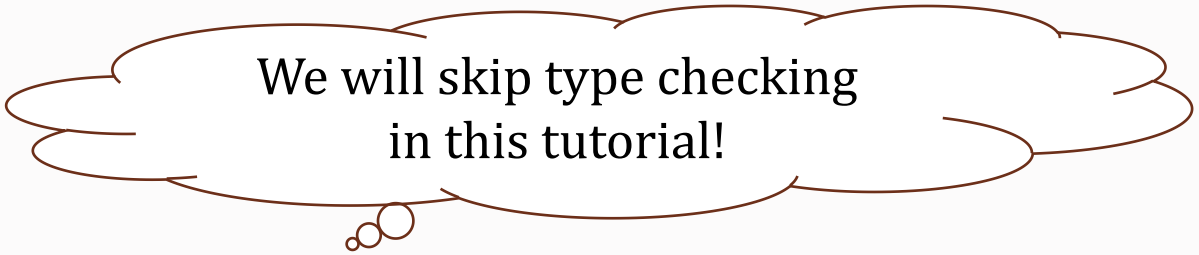
PART THREE

03 Evaluation

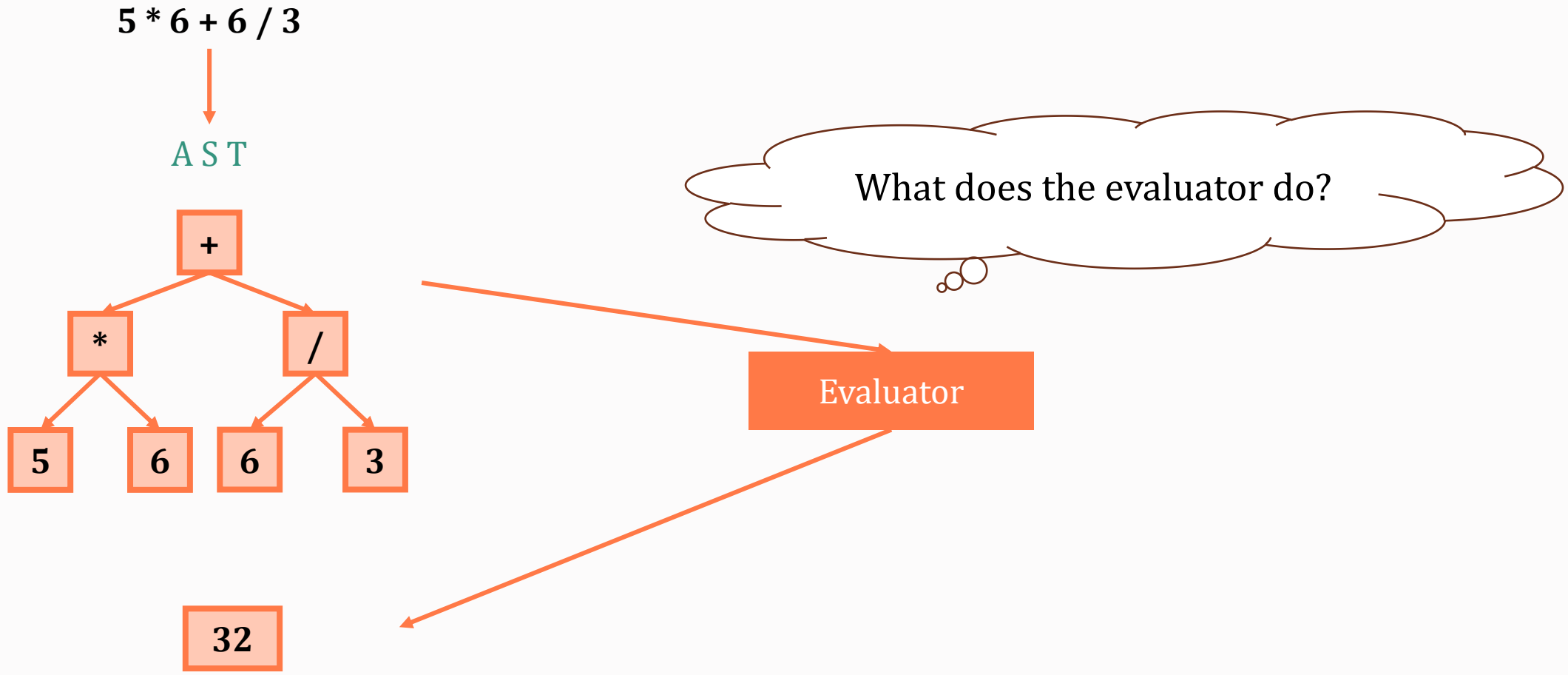


PART THREE

03 Evaluation



Evaluation

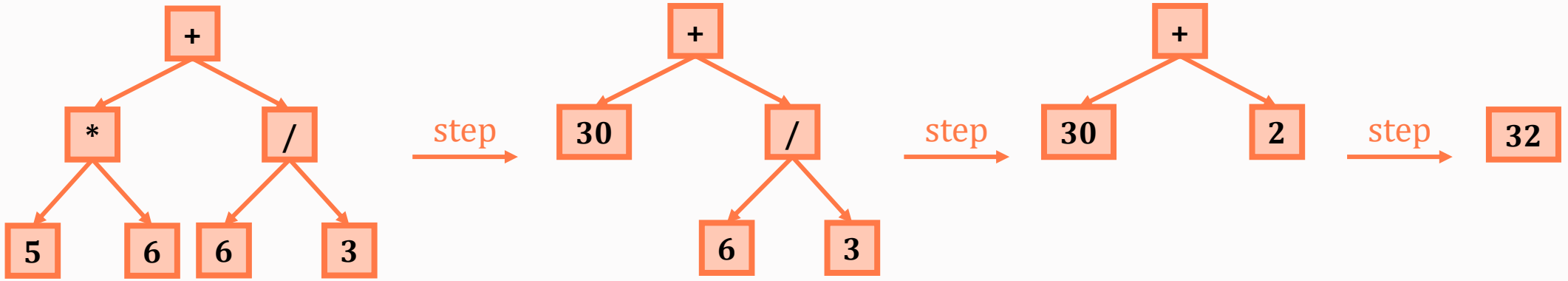


Evaluation

- Evaluation is the process of continuing to simplify the AST until it's just a ***value***. In other words, evaluation is the implementation of the language's dynamic semantics.
- The evaluator makes an *expression* `e` take a single step at a time to simplify itself, and `e` will be a *value* eventually after multiple steps. We call this kind of strategy "small-step" or "single-step", since we only step one step at a time.

```
e --> e1 --> .... --> en --> v
```

- As you may recall that our *expression* is recursively defined, and thus we need to recursively step the *expression*.



Evaluation

- Here is the BNF syntax of simple mathematical expressions:

```
e ::= i | e1 binop e2
binop ::= + | - | * | /
```

- We establish the following stepping rules (we use --> for a single-step):
  - **Constants:** Constants (integers) are already *values*, so we **cannot** step them further.
  - **Binary Operators:** A binary operator application `e1 bop e2` has two subexpressions, `e1` and `e2`. Since `e1` and `e2` can also be *values*, we need to consider three different circumstances:

```
I.   v1 bop v2 --> v
II.  v1 bop e2 --> v1 bop e2'
III. e1 bop e2 --> e1' bop e2
```

`e1 bop v2 --> e1' bop v2` indicates that as we step the whole expression, we actually recursively step `e1`.

You may notice that we don't consider `e1 bop v2 --> e1' bop v2`, because it is included in III.

- Note that we establish these rules based on that e1 and e2 are "*steppable*", which is true for operations + - * /. If not, e1 or e2 should be a value and consumed by earlier rules.
- Rule I stands for *primitive operation* processing, which is slightly different with II and III, indicating that we need not only syntaxes but **semantics** to do some "calculations".

Evaluation

- Now let's implement the single-step in our own interpreter.
- First, we need a function to check whether an *expression* is already a *value*, since we cannot step *values*. Or we can say that this function checks whether a *expression* is *steppable*.

In main.ml

```
(* check if an expression is a value (i.e., fully evaluated) *)
let is_value : expr -> bool = function
  | Int _ -> true
  | Binop _ -> false
```

Evaluation

- Next, let's implement the small-stepping rules.
  - Since we need to recursively step the expression, we need a recursive function.
  - Rule I and II need only syntaxes, while rule III need semantics as well. So, we need a special step function for it.

In main.ml

```
(* takes a single step of evaluation of [e] *)
let rec step : expr -> expr = function
  | Int _ -> failwith "Does not step on a number"

  (* No need for further stepping if both sides are already values *)
  | Binop (binop, e1, e2) when is_value e1 && is_value e2 ->
    step_binop binop e1 e2

  (* Evaluate the right side of the binop if the left side is a value *)
  | Binop (binop, e1, e2) when is_value e1 -> Binop (binop, e1, step e2)

  (* Leftmost step for binop *)
  | Binop (binop, e1, e2) -> Binop (binop, step e1, e2)
```

Evaluation

- Next, let's implement the small-stepping rules.
  - Since we need to recursively step the expression, we need a recursive function.
  - Rule I and II need only syntaxes, while rule III need semantics as well. So, we need a special step function for it.

In main.ml

```
(* implement the primitive operation [v1 binop v2].
   Requires: [v1] and [v2] are both values. *)
and step_binop binop v1 v2 = match binop, v1, v2 with
  | Add, Int a, Int b -> Int (a + b)
  | Sub, Int a, Int b -> Int (a - b)
  | Mul, Int a, Int b -> Int (a * b)
  | Div, Int a, Int b when b <> 0 -> Int (a / b)
  | Div, Int _, Int 0 -> failwith "division by zero"
  | _ -> failwith "precondition violated"
```

Evaluation

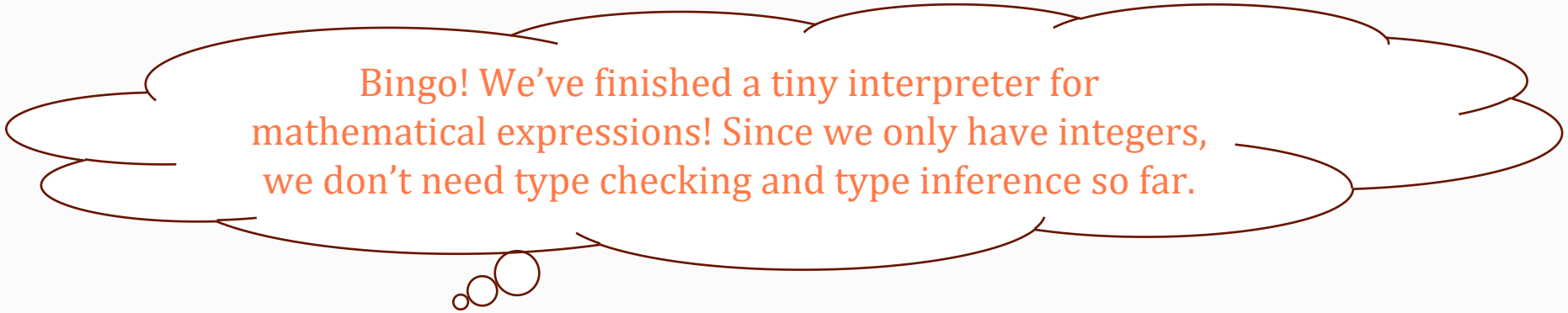
- Finally, we need to chain them up with an `eval` and an `interp` function.

In main.ml

```
(* fully evaluate [e] to a value [v] *)
let rec eval (e : expr) : expr =
  if is_value e then e else
    e |> step |> eval
```

In main.ml

```
(* interpret [s] by lexing -> parsing -> evaluating and converting the
result to a string *)
let interp ( s : string ) : string =
  s |> parse |> eval |> string_of_expr
```



Evaluation

- Hold on a second! You may notice that we mentioned the name of the stepping process above "single-step" or "small-step". But where's the "big-step"? Let's make a definition now.
- First, we define a symbol `-->*` to be the **reflexive transitive closure** of `-->`. We may call it multi-step. As you would have learned in Discrete Mathematics, it denotes that:

  - If `e -->.....--> e'` then `e -->* e'`
  - If `e -->.....--> v` then `e -->* v`
  - `v -->* v`

- Now, we can define big-step

  For all expressions `e` and values `v`, it holds that `e ==> v` if and only if `e -->* v`.

- We call `e ==> v` a "big-step" from *expression* `e` to *value* `v`, which is an abstraction of the whole small-step process: it just omits the intermediate steps, or even there is **no** step.

Evaluation

- Both two kinds of stepping have their own importance:
  - The small-step semantics tends to be easier to work with when it comes to modeling complicated language features.
  - The big-step semantics tends to be more similar to how an interpreter would actually be implemented.

Evaluation

- Now, let's establish rules for big-step:
  - **Constants**: Constants big-step to themselves:

```
i ==> i
```

  - **Binary Operators**: Binary operators just big-step both of their subexpressions, then apply whatever the primitive operator is:

```
e1 bop e2 ==> v
  if e1 ==> v1
  and e2 ==> v2
  and v is the result of primitive operation v1 bop v2
```

Evaluation

- Next, we can make implementations of function `eval_big` and `eval_bop`, while the former do the big-stepping evaluation and the latter do the semantic-related big-step for `Binop`. Also, chain them up with an `interp_big` function.

In main.ml

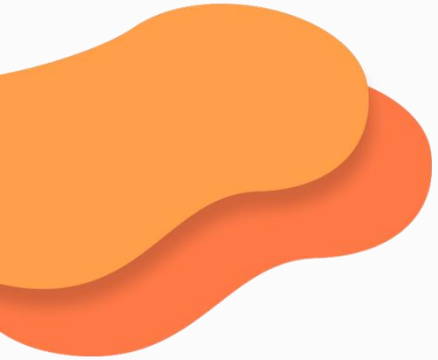
```
let rec eval_big (e : expr) : expr = match e with
  | Int _ -> e
  | Binop (binop, e1, e2) -> eval_bop binop e1 e2

and eval_bop binop e1 e2 = match binop, eval_big e1, eval_big e2 with
  | Add, Int a, Int b -> Int (a + b)
  | Sub, Int a, Int b -> Int (a - b)
  | Mul, Int a, Int b -> Int (a * b)
  | Div, Int a, Int b when b <> 0 -> Int (a / b)
  | Div, Int _, Int 0 -> failwith "Division by zero"
  | _ -> failwith "Operator and operand type mismatch“

let interp_big (s : string) : string =
  s |> parse |> eval_big |> string_of_expr
```

Evaluation

Finally! By far, we have added the big-step model into our Math Interpreter and finished its implementation. Let's implement an interpreter for a slightly more complex language – SimPL.



PART FOUR

04 SimPL Interpreter

SimPL

- As an example, we'll use a very simple programming language that we call SimPL. Here is its syntax in BNF. You can check it in SimPL.md of the release **SimPL Interpreter – Dev-Ready.**

```
e ::= x | i | b | e1 bop e2
      | if e1 then e2 else e3
      | let x = e1 in e2

bop ::= + | * | <=

x ::= <identifiers>

i ::= <integers>

b ::= true | false
```

- To put it simply, SimPL supports
  - **Values:** integers, Boolean and identifiers (consist of English letters).
  - **Binary Operators:** +(Add), *(Mul) and <=(Leq).
  - **Statements:** *let* statement and *if* statement.

SimPL Parsing

Task 1: Implement the parsing phase of SimPL. (20 min)

- Development-ready code is available in the release **SimPL Interpreter - Dev-Ready** of GitHub repo **AugustineYang/OCaml-Interpreter**.
- Your program should be able to parse the SimPL statements like:
  - `let x = 3110 in x + x`
  - `if x <= 3 then true else false`
- For example, the output of your parser parsing the first statement is expected to be something like this: `AST: Let (x, Int 3110, Binop (Add, Var x, Var x))`

- You are suggested to finish it on your own. But if you need some **hints**:
  1. Complete the AST according to SimPL syntax.
  2. Add lexing tokens and parsing rules.
  3. Add lexing rules.
  4. Complete the drivers like `string_of_expr` in main.ml.
- **You are suggested to finish it on your own.** The code is available in commit "Task 1: Implement parsing for SimPL" of branch *SimPL*. You can check it if you have any trouble.

SimPL Evaluation

Task 2: Implement the evaluator for *if* statement. (10 min)

- Our interpreter is unable to process *variables* like x. But don't worry. Your program only need to interpret the SimPL statements like:
  - `if 2 <= 3 then false else true`
- The output of your program interpreting the statement is expected to be something like this:

```
Result of interpreting test/simpl_test2.in:
Bool false

Result of interpreting test/simpl_test2.in with big-step model:
Bool false

AST: If (Binop (Leq, Int 2, Int 3), Bool false, Bool true)
```

- Design the small-step rules and big-step rules for *if*, just as what we did for binary operators.
- Since the *condition* expression of a *if* statement returns a Boolean value, and we use "<=" operator, you may also add some step-related rules for Boolean and "<=" operator.

SimPL Evaluation

- The small-step rules you designed for *if* expressions should be like:

```
I.   if true  then e2 else e3 --> e2
II.  if false then e2 else e3 --> e3
III. if e1    then e2 else e3 --> if e1' then e2 else e3
```

- Also, rule III indicates that e1 is *steppable*. If not, it should be either *true* or *false*, and it should have been consumed by rule I or rule II earlier.
- Here's what should be added in the small-step function. You can also do a type checking here.

```
(* takes a single step of evaluation of [e] *)
let rec step : expr -> expr = function
  | Int _ | Bool _ -> failwith "Does not step on a number"

  ......

  | If (Bool true, e2, _) -> e2

  | If (Bool false, _, e3) -> e3

  | If (Int _, _, _) -> failwith "Condition must be a boolean"

  | If (e1, e2, e3) -> If (step e1, e2, e3)
```

SimPL Evaluation

- You need to add some checks in function `is_value` for Boolean and *if* expression as well.

```
let is_value : expr -> bool = function
  | Int _ | Bool _ -> true
  | Binop _ | If _ | _ -> false
```

- Also, add a step rule for "<=" operator in function `step_binop`.

```
and step_binop binop v1 v2 = match binop, v1, v2 with

  ......

  | Leq, Int a, Int b -> Bool (a <= b)
```

- By far, you small-step evaluator for if expression should work! Now let's take a look at big-step evaluation.

SimPL Evaluation

- The big-step rules you designed for *if* expressions should be like:

```
I.   if e1 then e2 else e3 ==> v2
        if e1 ==> true
        and e2 ==> v2

II.  if e1 then e2 else e3 ==> v3
        if e1 ==> false
        and e3 ==> v3
```

- Here's what should be added in the big-step function. You can also do a type checking here if you like. Also, don't forget Boolean.

```
let rec eval_big (e : expr) : expr = match e with
  | Int _ | Bool _ -> e
  | Binop (binop, e1, e2) -> eval_bop binop e1 e2
  | If (e1, e2, e3) -> eval_if e1 e2 e3
```

```
and eval_if e1 e2 e3 = match eval_big e1 with
  | Bool true -> eval_big e2
  | Bool false -> eval_big e3
  | _ -> failwith "Condition must be a boolean"
```

SimPL Evaluation

- And, don't forget "<=" operator!

```
and eval_bop binop e1 e2 = match binop, eval_big e1, eval_big e2 with

  ......

  | Leq, Int a, Int b -> Bool (a <= b)
```

- By far, our evaluator is able to interpret simple *if* expression without variables. Complete code is available in commit "Task 2: Implement basic evaluation for if statement" of branch *SimPL*.

- Now let's take a look at *let* expressions where we have to deal with variables. Here comes the "**Substitution**".

PART FIVE

05 Substitution

Substitution

- **Substitution** is the process of replacing variables or expressions with their corresponding values or evaluated forms during execution. It is a fundamental mechanism for handling variable bindings, function applications, or macro expansions.
- Let's propose a new **notation** `e'{e/x}` meaning "the expression `e'` with `e` substituted for `x`." The intuition is that **anywhere `x` appears in `e'`, we should replace `x` with `e`.**
- For example, suppose we are evaluating `let x = 1 in x + 42`:

```
    let x = 1 in x + 42
--> (x + 42){1/x}
  = 1 + 42
--> 43
```

Substitution

- Now, we can make some small-step evaluation rules for *let* expressions:

```
I.   let x = v1 in e2 --> e2{v1/x}
II.  let x = e1 in e2 --> let x = e1' in e2
```

- Remember that, if `e1` in rule II is not a *steppable* expression, the *let* expression should have been consumed by rule I. That's the reason why we can safely step `e1`.

- The rules above also illustrate a fact that `x`, as a variable, is **not a value**, and also **unsteppable**. If we're trying to step a variable rather than do substitution on it, it means that the variable is unbound (i.e., didn't consumed by an appropriate substitution earlier), in which case an error should be raised. Like this simple OCaml code:

```
# let y = x in y;;
Error: Unbound value x
```

- It is a type-checking error actually. But we can leave the error type along for now, and just raise the error when the evaluator tries to step a variable.

Substitution

- Furthermore, we can make some big-step evaluation rules for *let* expressions:

```
let x = e1 in e2 ==> v2
  if e1 ==> v1
  and e2{v1/x} ==> v2
```

- Don’t forget that a variable is unsteppable, as we have explained in the previous slide. An error should be raised if the evaluator tries to step a variable.

```
x =/=>
```

SimPL Substitution

Task 3: Implement basic evaluation for *let* statement. (5 min)

- Now we can finish our evaluator for *let* statements. Complete small-step and big-step evaluator for *let* expression. Don't forget varaiables.
- You may notice that we only introduced substitution to our expressions, without doing the real substitution. Don't worry. Let's just define a `subst` function like this. We can safely call it and leave its details for next task.

```
(** [subst e v x] is [e{v/x}]. *)
let subst _ _ _ =
  failwith "TODO: implement substitution"
```

- The output of your interpreter interpreting `let x = 3110 in x + x` should be `Fatal error: exception Failure("TODO: implement substitution")`.

- **You are suggested to finish it on your own.** The code is available in commit "Task 3: Implement basic evaluation for let statement" of branch *SimPL*.

Substitution

Now we've already introduced substitution to SimPL with *let* expression. Let's take a look at what we should do for `e2{v1/x}` in `let x = v1 in e2 --> e2{v1/x}`. In other words, let's give a careful definition of substitution for SimPL expressions:

1. **Constants:** Integers and Booleans have no variables appearing in them, so substitution leaves them unchanged:

   ```
   i{e/x} = i
   b{e/x} = b
   ```

2. **Variables:** Either we encounter the same variable `x`, which means we should do the substitution, or we encounter some other variable with a different name, say `y`, in which case we should not do the substitution:

   ```
   x{e/x} = e
   y{e/x} = y
   ```

Substitution

3. **Binary operators:** We should recurse inside the subexpressions to do the substitution:

```
(e1 bop e2){e/x} = e1{e/x} bop e2{e/x}
```

4. **If expressions:** We should recurse inside just as what we do on binary operators:

```
(if e1 then e2 else e3){e/x} = if e1{e/x} then e2{e/x} else e3{e/x}
```

5. **Let expressions:** For let expressions, things get tricky. Suppose we have two more complex situations:

```
    let x = 5 in let x = 6 in x
--> (let x = 6 in x){5/x}
  = let x = 6{5/x} in x
  = let x = 6 in x
--> x{6/x}
  = 6
```

```
    let x = 5 in let y = 1 + x in y * x
--> (let y = 1 + x in y * x){5/x}
  = let y = (1 + x){5/x} in (y * x) {5/x}
  = let y = 6 in y * 5
--> (y * 5){6/y}
  = 30
```

Substitution

5. **Let expressions:** For let expressions, things get tricky. Suppose we have two more complex situations:

```
    let x = 5 in let x = 6 in x
--> (let x = 6 in x){5/x}
  = let x = 6{5/x} in x
  = let x = 6 in x
--> x{6/x}
  = 6
```

```
    let x = 5 in let y = 1 + x in y * x
--> (let y = 1 + x in y * x){5/x}
  = let y = (1 + x){5/x} in (y * x){5/x}
  = let y = 6 in y * 5
--> (y * 5){6/y}
  = 30
```

See the difference? We have two different strategies, depending on the name of the **bound variable.** So here's the substitution definition for *let* expressions.

```
(let x = e1 in e2){e/x}  =  let x = e1{e/x} in e2
(let y = e1 in e2){e/x}  =  let y = e1{e/x} in e2{e/x}
```

Substitution

Here are some extra examples for you to understand the substitution on *let* expressions better. You can check them out after class.

```
    let x = 2 in x + 1
--> (x + 1){2/x}
  = 2 + 1
--> 3
```

```
    let x = 0 in (let x = 1 in x)
--> (let x = 1 in x){0/x}
  = (let x = 1{0/x} in x)
  = (let x = 1 in x)
--> x{1/x}
  = 1
```

```
    let x = 0 in x + (let x = 1 in x)
--> (x + (let x = 1 in x)){0/x}
  = x{0/x} + (let x = 1 in x){0/x}
  = 0 + (let x = 1{0/x} in x)
  = 0 + (let x = 1 in x)
--> 0 + x{1/x}
  = 0 + 1
--> 1
```

Task 4: Implement substitution. (15 min)

- Now you can finish your `subst` function with the definitions above. And your interpreter will be able to use variables. Have a try!
- Here are two samples with their corresponding outputs for you to check your interpreter.

1. `let x = 3110 in x + x`

```
Result of interpreting test/simpl_test1.in:
Int 6220

Result of interpreting test/simpl_test1.in with big-step model:
Int 6220

AST: Let (x, Int 3110, Binop (Add, Var x, Var x))
```

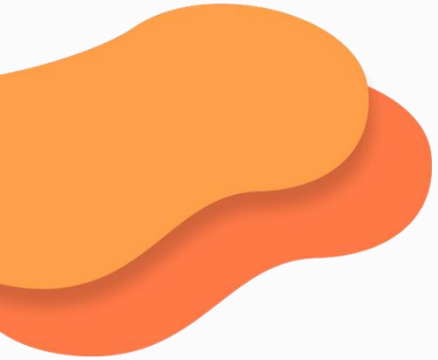
2. `let y = 6 in let x = 5 * 6 in if x <= 30 - y then x - 1 else y + 1`

```
Result of interpreting test/simpl_test2.in:
Int 7

Result of interpreting test/simpl_test2.in with big-step model:
Int 7

AST: Let (y, Int 6, Let (x, Binop (Mul, Int 5, Int 6), If (Binop (Leq, Var x, Binop (Sub, Int 30, Var y)), Binop (Sub, Var x, Int 1), Binop (Add, Var y, Int 1))))
```

- **You are suggested to finish it on your own.** The code is available in commit "Task 4: Implement substitution" of branch *SimPL*.



PART SIX

06

Lambda Calculus Interpreter

Lambda Calculus

- Consider this tiny language:

```
e ::= x | e1 e2 | fun x -> e
v ::= fun x -> e
x ::= <identifiers>
```

- Seems familiar? This syntax is exactly the *lambda calculus*. There are only three expressions in it: varaibles, function application, and anonymous function. The only values are anonymous functions. Though this syntax is not even typed, it can express any computable function. You should have learnt the knowledge from earlier lectures.

```
# (fun y -> y + 1) 5;;
- : int = 6
```

- Let's add the support for lambda calculus syntax into our interpreter.

Lambda Calculus Evaluation

- There are several ways to define an evaluation semantic for the lambda calculus. Here's a rule that really close to OCaml one:

```
e1 e2 ==> v
  if e1 ==> fun x -> e
  and e2 ==> v2
  and e{v2/x} ==> v
```

- Remember? This is the **call by value** semantics covered in previous lectures! It requires arguments to be reduced to *values* before a function can be applied.
- Let's try to define the **call by name** semantics.

```
e1 e2 ==> v
  if e1 ==> fun x -> e
  and e{e2/x} ==> v
```

- In **call by name** semantics, `e2` does not have to be reduced to a *value*. It can lead to great efficiency if the value of `e2` is never needed.

Lambda Calculus Substitution

- Now we need to define the substitution operation for the lambda calculus. We can make a definition that works for either call-by-name or call-by-value. Inspired by the SimPL substitution, there should be quite little difficulty making these rules. Here's the beginning of our definition:

```
x{e/x} = e
y{e/x} = y
(e1 e2){e/x} = e1{e/x} e2{e/x}
```

- What about the functions? In SimPL, we stopped substitution when we reached a bound variable of the same name; otherwise, we proceeded. In lambda calculus, maybe we can do the same. Let's have a try:

```
(fun x -> e'){e/x} = fun x -> e'
(fun y -> e'){e/x} = fun y -> e'{e/x}
```

Lambda Calculus Substitution

- Now we need to define the substitution operation for the lambda calculus. We can make a definition that works for either call by name or call by value. Inspired by the SimPL substitution, there should be quite little difficulty making these rules. Here's the beginning of our definition:

```
x{e/x} = e
y{e/x} = y
(e1 e2){e/x} = e1{e/x} e2{e/x}
```

- What about the functions? In SimPL, we stopped substitution when we reached a bound variable of the same name; otherwise, we proceeded. In lambda calculus, maybe we can do the same. Let's have a try:

```
(fun x -> e'){e/x} = fun x -> e'
(fun y -> e'){e/x} = fun y -> e'{e/x}
```

Correct?

Lambda Calculus Substitution

```
(fun x -> e'){e/x} = fun x -> e'
(fun y -> e'){e/x} = fun y -> e'{e/x}
```

Correct?

- Consider these two expressions, both of which should ignore arguments and return x (i.e., the value assigned to x):

```
I.      let x = z in (fun y -> x)
    --> (fun y -> x) {z/x}
      = (fun y -> z)

II.     let x = z in (fun z -> x)
    --> (fun z -> x) {z/x}
      = (fun z -> z)
```

Lambda Calculus Substitution

```
(fun x -> e'){e/x} = fun x -> e'
(fun y -> e'){e/x} = fun y -> e'{e/x}
```

Correct?

- Consider these two expressions, both of which should ignore arguments and return x (i.e., the value assigned to x):

```
I.      let x = z in (fun y -> x)
    --> (fun y -> x) {z/x}
      = (fun y -> z)
```

Expression I ignores the argument and returns z, which is bound to x. The result is correct.

```
II.     let x = z in (fun z -> x)
    --> (fun z -> x) {z/x}
      = (fun z -> z)
```

Expression II becomes a identity function that returns whatever it passed to it, rather than returning x's value. Something goes wrong.

Capturing

- Reconsider the second example. Here we colour different z with different colours:

```
II.     let x = z in (fun z -> x)
    --> (fun z -> x) {z/x}
      = (fun z -> z)
```

- Oops! The z disappeared! This is the place where things go wrong.
- Actually, the disappearance of z is called **capturing**. The broken substitution **captures** z and make it be the same variable as z. It breaks the variable scope. This is how substitution fails its job.
- Therefore, we need to enhance our substitution to avoid capturing. That's why we need **capture-avoiding substitution**.

Capture-Avoiding Substitution

```
(fun x -> e'){e/x} = fun x -> e'
(fun y -> e'){e/x} = fun y -> e'{e/x}
    if y is not in the free variables of e
```

- For the first rule, we stay the same. However, for the second one, we add a side-condition saying that "if y is not in the **free variables** of e".
- Recall the meaning of **free variables**. If we denote `FV(e)` as "the free variables of `e`", i.e., the variables that are not bound in it, we can make definitions as follows:

```
FV(x) = {x}
FV(e1 e2) = FV(e1) + FV(e2)
FV(fun x -> e) = FV(e) - x
```

- We have already learnt the definition of FV from both previous lectures and Discrete Mathematics.

Capture-Avoiding Substitution

```
(fun x -> e'){e/x} = fun x -> e'
(fun y -> e'){e/x} = fun y -> e'{e/x}
    if y is not in the free variables of e
```

- However, the side effect of this side-condition is that, substitution stuck when it encounters a y who is already in the FV of e. We have to deal with it.
- The solution is that, if ever about to capture a variable, replace it with a **fresh** name:
  1. About to capture z: `(fun z -> x) {z/x}`
  2. Replace argument: `(fun z1 -> x) {z/x}`
  3. It's safe now: `(fun z1 -> z)`
- We can use rules like z1, z2, z3, ... to create a fresh name. We have to maintain a list for all used names, and double-check that the new name is indeed fresh before using.

```
(** [gensym ()] generates a fresh variable name. *)
let gensym =
  let counter = ref 0 in
  fun () -> incr counter; "$x" ^ string_of_int !counter
```

Task 5: Implement interpreter for lambda calculus.

- Implement *parsing*, *evaluation* and *capture-avoiding substitution* for lambda calculus based on the SimPL interpreter. You can refer to the *syntax*, *evaluation rules* and *substitution rules* in the earlier slides. For simplicity, we only need to implement big-step evaluation. You can use either *call-by-value* or *call-by-rule* strategy, or both of them if you like.
- You may modify the structure of AST a little bit. You may use menhir instead of ocamlyacc to build parser.mly for some features. You may add some drivers in main.ml for new features.
- Here are a few samples for you to verify your interpreter:
  1. `(fun x -> x) (fun y -> y)`
  2. `(fun x -> fun x -> x) (fun a -> fun b -> a) (fun a -> fun b -> b)`
  3. `((fun x -> (fun z -> x)) z) (fun x -> x)`
  4. `(fun y -> y + 1) 5`
  5. `(fun x -> let y = 6 in let x = 5 * 6 in if x <= 30 - y then x - 1 else y + 2) 10`
- Your interpreter should reduce sample 1 and sample 2 to `fun y -> y` and `fun a -> fun b -> b` respectively, raise an *"unbound value" error* for sample 3, and interpret sample 4 and sample 5 to *6* and *8*.
- **You are suggested to finish it on your own.** The code is available in commit "Task 5: Implement interpreter for SimPL with lambda calculus" of branch *lambda*.

References

- Pierce, B. C. (2002). Types and Programming Languages. MIT Press.
- Clarkson, M. R. (2021–2025). [OCaml Programming: Correct + Efficient + Beautiful](https://cs3110.github.io/textbook/)

Thank you!